
Paradigmas de Programación

EIF400-II-2026- CLoria

Escuela de Informática – Universidad Nacional (UNA)

Proyecto Grupal: *Proyecto Cyphail*

Versión 0.1 Julio-2026

Proyecto Consultas sobre Grafos (nombre clave `Cyphail`)

0. Introducción

- Este documento constituye el `SPEC` principal del proyecto del curso el que, descrito en forma breve, **es una aplicación prototipo de manejo de grafos** cuyo nombre es `Cyphail` y cuyos requerimientos y arquitectura se perfilan más adelante.
- Durante el avance del proyecto podrán darse cambios o anexos a este SPEC según el proyecto lo requiere
- Un poco más específicamente y alineado con los objetivos del curso en forma transversal, `Cyphail` es un `compilador` (o más precisamente un `transpilador`) de un lenguaje de consulta a grafos junto con el correspondiente motor (`engine`) de ejecución.

Relación con la Carta al Estudiante (CE)

`Cyphail` corresponde al rubro de `Proyectos` programa del curso. Habrán dos sprints largos (`P1` y `P2`) y sprints cortos (`P1.1`, `P2.1`), según el avance del semestre lo permita.

Relación con Rubro de Investigación

Como parte del proyecto y para cumplir con el rubro de `Investigación` que la CE del curso también incluye, hay un `SPEC` de la investigación sobre el tema de lenguajes tipo `Cypher` que los estudiantes desarrolladores del proyecto deben realizar y sin la cual no es posible llevar a cabo la aplicación `Cyphail`. Ese documento referencia a este `SPEC`.

Metodología Ágil

Conforme con el enfoque iterativo del curso `Cyphail` será un proyecto de tipo **ágil**.

- Los alcances de cada sprint se irán definiendo según el avance del curso.
- Es un trabajo grupal en los grupos formados oficialmente.
- Asimismo, podrán haber anexos a este `SPEC` que varíen, eliminen o añadan requerimientos según el desarrollo del curso lo demande.

Manejo del Proyecto

- Se espera el proyecto sea manejado siguiendo formas de trabajo estándares y profesionales de manejo de proyectos.

- Se espera una estructura de proyecto manejada para trabajo compartido, tipo `git/github` (o equivalente), con su `Readme`, build eficaz, documentación del código, casos de prueba, estilos y patrones que representen los paradigmas estudiados.
- Cada funcionalidad obligatoria debe acompañarse con casos de prueba unitarios y de integración que son incorporados al testing. Se entienden estos como **parte obligatoria** del entregable. El grupo reconoce esto así e incluye en su trabajo crear esos casos de prueba e incluirlos en el proceso de build.
- El equipo se organiza para cumplir con esas expectativas, pero sin que algún miembro pierda conexión con alguna parte del proyecto, bajo el criterio que **cualquiera es capaz de defender el proyecto en forma individual**.

Fechas de Entrega

- Las formas y fechas concretas de entrega se definirán oportunamente.
- Se planea lo siguiente:
 - El primer sprint corto `P1.1` para la semana 6.
 - El primer sprint largo `P1` esté en la semana 9
 - El segundo sprint corto `P2.1` para la semana 13.
 - El sprint final `P2` en semana 16.
- Expectativa P1.1: es un setup inicial formal del proyecto. Incluye el `REPL` y finge la existencia de la conexión con Prolog.
- Expectativa P1: Lexer/Parser. Interacción con Prolog y faked mensajes (JSON)
- Expectativa P2.1: Motor en Prolog con casos simples
- Expectativa P2: Motor en Prolog con casos extendidos

Forma de Entrega

- El coordinador de cada grupo sube un `zip` (o equivalente) al drive que se le asigne al grupo de trabajo por el docente en su momento.
- El `zip` **debe cumplir estrictamente** con las formas, convenciones y tiempos de entrega que se le pidan oportunamente. No cumplirlas puede hasta anular el entregable.

Revisiones y Defensa

- Habrá una demo/defensa para cada entregable (sprint) en horas de clase de la funcionalidad obtenida.
- El profesor revisará no solo funcionalidad sino también calidad de código, según su criterio, en base a lo aprendido en el curso. Que funcione puede no ser suficiente, factores como documentación, legibilidad y apego a paradigmas pedidos cuentan igualmente.
- Toda defensa debe poder realizarse **sin un ambiente de desarrollo específico** (IDE o similar). **Solo una consola** debe ser necesaria y **siguiendo fielmente las instrucciones que el `Readme` del proyecto indique para construir y correr la aplicación**.
- En general, cualesquiera **dificultades ajenas al profesor** para revisar y probar un entregable anulan el mismo.

- Durante una defensa, el profesor escoge a un estudiante del grupo como el `conductor` en turno y solo esa persona puede responder a lo pedido a nombre del grupo. La evaluación del entregable puede ser ajustada según la capacidad del conductor de defender el trabajo. El profesor puede cambiar el `conductor` varias veces o mantener uno solo.
- La revisión de código requiere que cada fuente tenga los autores y el grupo trabajo. Asimismo, se revisa el apego al paradigma esperado. Si una solución se esperaba en un paradigma y no se cumple eso provocará pérdida de puntos o anulación del todo.

Use de *Librerías* de Terceros

- Se deberán usar los lenguajes, paradigmas y herramientas que se autoricen por el docente.
- El uso de *librerías* o herramientas debe ser autorizado por el docente. Lo contrario puede anular un entregable.

Uso de IA

- Está prohibido el uso de cualquier `IA` agéntica (agentic *coding*) que genere el proyecto *zero coding* (es decir, no se permite `Claude Code` o comparables). Se considera como copia y/o plagio.
- El uso de la `IA` para **entender** y **estudiar** es **autorizada** y **promovida**. Pero el estudiante debe poder defender cualquier producto de su grupo de trabajo como si fuera propio.
- Un uso autorizado puede requerir mostrar los prompts que fueron empleados así como el modelo (`LLM`) usado.

Extras

- Un `extra` es alguna funcionalidad adicional a las obligatorias de un entregable que un grupo decide querer incluir voluntariamente
- Puede ser originada por el grupo mismo o sugerida por el profesor
- En cualquier caso debe ser autorizada por el profesor para ser aceptable.
- Solo suma a la nota si el proyecto cumple con al menos un 95% de lo obligatorio. No puede ser más de un 10 puntos (máxima nota 110 puntos de 100).

1. Objetivo del Prototipo

Construir, a lo largo del semestre, un **motor de consultas sobre grafos en memoria**, para un lenguaje propio inspirado en `Cypher` (el lenguaje de `Neo4j` y otras herramientas de manejo de grafos), capaz de:

- Construir, modificar y persistir un grafo de datos (`CREATE` , `SET` , `DELETE` , `SAVE` , `LOAD`).
- Consultarlo mediante pattern-matching de nodos y relaciones (`MATCH` , `WHERE` , `RETURN`).
- Ejecutar dichas consultas usando **SWI-Prolog como máquina de ejecución real**, no una simulación.

Ventajas de Grafos

- Los grafos y casos de prueba son típicamente ejemplificados con una red social simplificada (personas, relaciones como `FOLLOWS` , `LIKES`).
- Usualmente son modelos sin un esquema rígido o del todo, lo cual les da dinamismo ante cambios en tiempo real.

- Un grafo es intuitivo, no requiere explicación de dominio, y es fácil de extender con propiedades arbitrarias.

Alcances Generales de Proyecto

- El proyecto no busca replicar `Cypher/openCypher` completo, solo busca ser un vehículo de aprendizaje donde **cada paradigma del curso (OOP, FP, LP) resuelve una parte interesante del proyecto**, de modo que ningún paradigma tenga un rol y utilidad en el proyecto.
- Los alcances específicos se determinan basados en los caso de uso identificados en la siguiente sección y se basan en la sintaxis del dialecto `Cyphail` que se ajusta a la de `Cypher`.
- No se descarta que se pida cambiar la gramática definitiva de `Cypher` sin alterar la semántica.

2. Casos de Uso según Gramática Objetivo

#	Categoría	Ejemplo	Requerimiento
1	Match simple	<code>MATCH (p:Person) RETURN p.name</code>	Recorrido + proyección
2	Match con relación	<code>MATCH (p:Person)-[:FOLLOWS]->(q:Person) RETURN p.name, q.name</code>	Unificación real sobre aristas
3	Filtro	<code>[...] WHERE p.age > 25</code>	Filtrado funcional de bindings
4	Agregación	<code>RETURN p.name, COUNT(q) AS seguidos</code>	<code>reduce</code> /agrupación
5	Múltiples patrones (join implícito)	<code>MATCH (a)-[:FOLLOWS]->(b), (b)-[:FOLLOWS]->(c) WHERE a <> c</code>	Backtracking con puntos de decisión
6	Negación	<code>WHERE NOT (p)-[:FOLLOWS]->()</code>	Negación por falla (<code>NAF</code>)
7	Multi-hop acotado (opcional)	<code>MATCH (a)-[:FOLLOWS*1..3]->(b)</code>	Recursión declarativa
8	Camino más corto (opcional)	<code>MATCH path = shortestPath((a:Person {name: "Ana"}) -[:FOLLOWS*]-> (b:Person {name: "Elena"})) RETURN path</code>	Lograr balance entre lo declarativo y procedural en un BFS. Nota: <code>más corto</code> es respecto del largo del camino.
9	Creación de nodo	<code>CREATE (a:Person {name: "Ana", age: 30})</code>	Escritura, mapas de propiedades
10	Creación de relación	<code>CREATE (a)-[:FOLLOWS]->(b)</code>	Referencia a nodos ya ligados

#	Categoría	Ejemplo	Requerimiento
11	Match + Create combinado	<code>MATCH (a), (b) WHERE id(a)=1 AND id(b)=2</code> <code>CREATE (a)-[:FOLLOWS]->(b)</code>	Identidad de nodo (<code>id()</code>) como concepto de lenguaje
12	Actualización de propiedades	<code>MATCH (p:Person {name:"Ana"}) SET p.age = 31</code>	"Update" declarativo — en Prolog es retract + assert, no mutación in-place
13	Borrado simple	<code>MATCH (p:Person {name:"Ana"}) DELETE p</code>	Debe fallar si el nodo tiene relaciones
14	Borrado en cascada	<code>MATCH (p:Person {name:"Ana"}) DETACH DELETE p</code>	Borra el nodo y todas sus relaciones
15	Persistencia	<code>SAVE "grafo.cypher" / LOAD "grafo.cypher"</code>	Statements propios del lenguaje (no comandos de sesión), vía unparser simétrico al parser

Alcances Específicos

- **Alcance Obligatorio:** casos 1–6, 9–15.
- **Extensión Posible (cierre de semestre):** casos 7–8. Posible quiere decir, que podrían volver obligatorios dependiendo del avance del curso.
- **Fuera de alcance:** `MERGE` (upsert) y `REMOVE` (quitar propiedad sin borrar el nodo) pues son variantes menores de `CREATE` / `SET` quedan como posible extra.

3. Fundamentos Teóricos y Herramientas

Conceptos por Paradigma

- En la tabla siguiente se usa `OOP` por programación orientada a objetos, FP para programación funcional y LP para programación lógica.
- Hay otros acrónimos y términos que serán explicados durante las clases.
- No es esencial entenderlos todos al arrancar el curso, obviamente. El objetivo es dar una idea de cómo se relaciona el curso con el proyecto según temática.

Paradigma	Conceptos que el proyecto obliga a aprender
OOP (repaso, ya lo conocen)	Encapsulamiento del estado del grafo antes de compilarlo; gestión del ciclo de vida de un proceso externo (conexión <code>MQI</code>)
FP (énfasis principal)	

Paradigma	Conceptos que el proyecto obliga a aprender
	Tipos algebraicos (<code>sealed</code> + <code>record</code>), pattern matching exhaustivo, parser combinators, inmutabilidad disciplinada, pipelines <code>map</code> / <code>filter</code> / <code>reduce</code> , el par simétrico parser/unparser, decodificación de JSON dinámico vía modelo de árbol + pattern matching
LP (énfasis esencial)	Unificación, resolución SLD, backtracking y puntos de decisión, negación como falla (asunción de mundo cerrado), recursión declarativa, <code>retract</code> / <code>assert</code> como modelo de "actualización" sobre hechos inmutables
Transversal	El <i>Expression Problem</i> : mismo problema (despacho por tipo de nodo del <code>AST</code>) resuelto como Visitor (OOP) vs. <code>switch</code> sobre sealed types (FP)

Herramientas y Stack de Desarrollo

- ****Java 26+ ****: requerido para *sealed interfaces*, *record patterns* y *pattern matching for switch*, todos finalizados (no preview) desde la versión (21).
- **SWI-Prolog** con `library(mqi)` : máquina de ejecución real vía *Machine Query Interface* (protocolo TCP + JSON, framing por longitud, comandos `run` / `close` / `quit`). Nota de plataforma: en Windows, el framing puede llegar como `\r\n` en vez de `\n` por el modo de traducción de saltos de línea de los streams de texto de SWI-Prolog (`SIO_NL_DOS`): el broker debe tolerar ambos.
- **Gson**: usado en modo *árbol* (`JsonElement`), no en modo POJO anotado pues la respuesta de MQI no tiene forma fija, implicando que deconstruirla es un ejercicio de pattern matching (FP), no de data-binding automático.
- JUnit (o PIUnit análogo en SWI-Prolog) para pruebas de el `CLI` , `parser` , `codegen` y de las consultas `end-to-end` .

Tipo de Aplicación y Uso

El entregable contiene una aplicación de consola (`CLI`) llamada `cyphail` . Esta recibe un comando y opciones del mismo, de la forma usual. Los comandos básicos son `help` , `run` y `compile` . Pueden haber otros. Cada comando puede tener opciones de la manera usual en ambientes IX (opciones posicionales, `--` o `-`)

Ejemplos:

- Para ejecutar un script de `Cyphail` llamado `movies` ubicado en una carpeta `examples` del directorio actual:

```
cyphail run ./examples/movies.cyphail
```

- Para interactuar con el motor en un shell (p repl)

```
cyphail repl
```

- Para ver el código generado a partir de un script `./src/movies` y dejar la salida en `./target/movies.pl` :

```
cyphail compile ./src/movies.cyphail --out ./target/movies.pl
```

- Para ver la ayuda de un comando (en este caso compilar):

```
cyphail help compile
```

4. Arquitectura

```

Aplicación host (Java OOP + FP)
|- UI Controller: Implementa la interacción con el usuario
|- Graph builder (OOP): construye/valida el grafo antes de compilarlo
|- Parser + codegen (FP basado)      - texto Cyphail -> AST (sealed+record) -> texto Prolog
|- PrologEngine / broker MQI (OOP) - sockets, framing, handshake, timeout configurable
|- Decoder JSON->PrologTerm (FP) - Gson en modo arbol + pattern matching, mismo patron que el AS
|- Pipeline de salida (FP)      - despacha por tipo de Statement: tabla / confirmacion / mensaje

Proceso SWI-Prolog (LP)
|- Hechos dinamicos      - node/2, has_property/3, edge/3 (generados por CREATE/SET/DELETE)
|- Libreria de soporte .pl - escrita a mano: negación, path_len/3 recursivo, bfs/3

```

Estrategias de Decisiones de Diseño

- El siguiente es un recuento de estrategias cuya comprensión requiere asimilar primero la materia del curso sobre temas de compilación, `SWI-Prolog`, modelaje `OOP-FP` y las que el estudiante investigará por si mismo sobre `Cypher` (mediante el trabajo de investigación especificado en documento aparte). Por ende, se recomienda usar estas decisiones en **forma incremental** según el avance del proyecto y el entregable del sprint en desarrollo.

Decisiones Generales: Diseño e Implementación

- El proyecto principal es en `Java` puede ser un proyecto `maven` o `gradle`. Pero recuerde que las defensas y pruebas del profesor son **sin IDE** de apoyo, solo consola. La escogencia libre del IDE es decisión de cada grupo, siempre que **no incumplan las restricciones de uso de IA**.
- El manejo de argumentos (comandos y opciones) por línea de comando de consola (comando `cyphail`) queda en `Java` no en `.bat` o equivalente. No vale la pena hacerlo desde cero, se usará `picocli` que además ilustra el patrón de decorador usando anotaciones.
- El lexer/parser se construye *"a mano"* usando combinadores de parsing (`Parser<T>` como interfaz funcional con combinadores como `map`, `flatMap`, or, `many`). No se usará ningún generador de parsers (`ANTLR4`, `JavaCC` u otro). El tokenizado de bajo nivel puede apoyarse en `regex` de `Java` estándar.
- El tokenizador trata el espacio en blanco (, `\t`, `\n`, `\r`) como separador implícito entre tokens y lo descarta antes de llegar al parser de combinadores. Esto a diferencia de la gramática formal de `openCypher`, que lo declara como símbolo explícito en cada producción. **Excepción:** el espacio en blanco se preserva dentro de literales de string y comentarios.
- El pipeline de salida distingue primero éxito ó fallo al resolver un estatuto. Dependiendo de esto:
 - Si es exitoso se muestra una tabla por defecto para aquellos con `RETURN` y un mensaje de confirmación para `CREATE`, `SET`, `DELETE`, `DETACH DELETE`, `SAVE`, `LOAD`.
 - Cualquier error (de sintaxis, de Prolog, o de timeout) se muestra como mensaje de error no como una tabla vacía o mal formada.
 - Recomendación de la `IA` **no vinculante** para el formato de tabla: `com.github.freva:ascii-table` pero cualquier librería equivalente y bien mantenida es aceptable.
- El grafo vive en `SWI-Prolog`, no en `Java`, lo que mantiene el modelo en `Java` usando estilos/patrones `OOP` bien conocidos por el estudiante y que extenderemos con estilos `FP` que se estudian en el curso.

- Las propiedades se representan de forma genérica (`has_property(NodeId, Key, Value)`), no un predicado por propiedad. Esto permite tener esquemas arbitrarios definidos por `CREATE`.
- La identidad de nodo (`id()`) es un concepto del **lenguaje**, no del backend. La implementación usará *símbolos generados* de Prolog, pero la semántica del lenguaje no depende de eso.
- Una consulta es una **secuencia de cláusulas que transforman un *environment* de bindings** (`MATCH → WHERE → SET/DELETE/CREATE → RETURN`). Las cláusulas de escritura son las únicas con efecto lateral obligatorio (`assertz` / `retract`). Acá hay dos opciones: transformación **immutable** (bindings como listas) o **mutable** (bindings como hechos (facts)).
- `SET` no es mutación *in-place*: se compila a `retractall` + `assertz`, porque los hechos Prolog son inmutables, es decir, "actualizar" es por debajo "reemplazar".
- `DELETE` sin `DETACH` de un nodo debe fallar si este tiene relaciones para mantener integridad referencial.
- `SAVE` / `LOAD` son propuestas particulares de este proyecto para tratarlas como statements del lenguaje (no como comandos de sesión del REPL): `SAVE` es un *unparser* (`AST → texto Cyphail`), función pura, simétrica del parser). Y `LOAD` reutiliza el mismo pipeline de ejecución sin requerir un componente nuevo.
- El timeout de cada `run(Goal, Timeout)` **no debe quedar fijo en `-1`** — un valor por defecto razonable (5-10s) convierte bugs de recursión infinita (ej. `path_len/3` mal acotado) en errores claros de timeout en vez de un REPL colgado.
- Solo se implementan 3 de los 6 comandos de MQI (`run`, `close`, `quit`) — no se necesita ejecución asíncrona para un REPL interactivo.
- Para el manejo de respuestas JSON del motor (o para cualquier uso de `JSON`) se usa `Gson`.
- El decoder de respuestas usa el modelo de árbol de `Gson` (`JsonElement`), no data-binding a POJOs anotados, porque la forma de la respuesta varía según la consulta.
- Los meta-predicados/combinadores de `Prolog` (`map`, `findall`, `aggregate_all`, `forall`, etc.) se deben usar en los casos donde la recursión solo estaría simulando iteración sobre una colección ya conocida (como en proyección de bindings, escritura masiva de hechos en CREATE/DETACH DELETE). Se preserva la recursión/backtracking explícita en los predicados de búsqueda donde el espacio de soluciones no se conoce de antemano y la recursión es un mecanismo real no una opción de estilo.